

Nexora E-Commerce Application on Azure

3-Tier Architecture Deployment Documentation

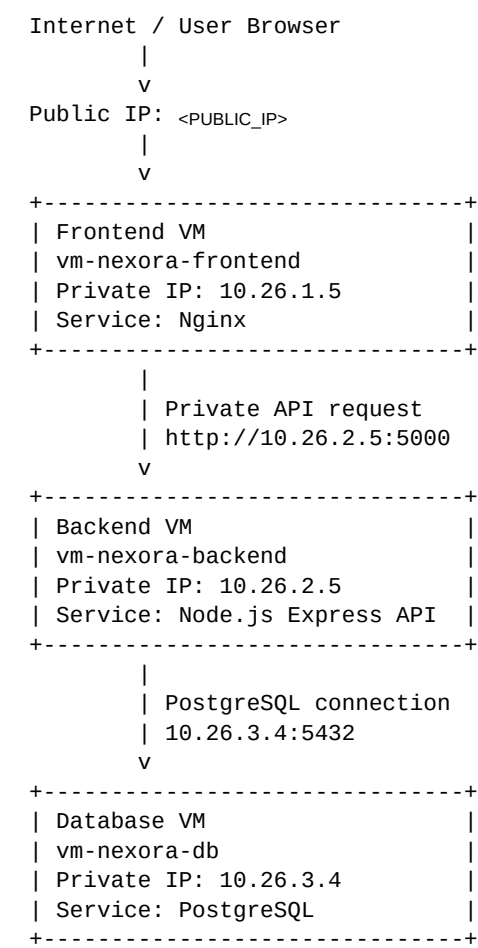
Frontend: Nginx | Backend: Node.js Express | Database: PostgreSQL | Cloud: Microsoft Azure

1. Project Overview

This document explains the deployment of Nexora, a sample e-commerce application hosted on Microsoft Azure using a secure 3-tier architecture. The application separates the frontend, backend API, and database into different virtual machines and subnets.

The frontend VM serves the website through Nginx and is the only tier exposed to the internet. The backend VM runs a Node.js Express API and communicates privately with the PostgreSQL database VM. The database tier stores product data and is accessible only through private networking.

2. Final Architecture



3. Azure Resources

Resource Type	Resource Name / Value
---------------	-----------------------

Resource Group	rg-nexora-p26
Virtual Network	vnet-nexora-p26
Frontend VM	vm-nexora-frontend
Backend VM	vm-nexora-backend
Database VM	vm-nexora-db
Frontend Public IP	<PUBLIC_IP>
Frontend Private IP	10.26.1.5
Backend Private IP	10.26.2.5
Database Private IP	10.26.3.4

4. Network Design

Subnet	CIDR Range	Purpose
frontend-subnet	10.26.1.0/24	Public-facing web tier
backend-subnet	10.26.2.0/24	Private backend API tier
db-subnet	10.26.3.0/24	Private database tier

The frontend tier receives public traffic. The backend and database tiers communicate using private IP addresses inside the Azure virtual network.

5. Access and Security Model

Only the frontend VM is exposed to the internet. Backend and database VMs are accessed through private IP addresses. This reduces the public attack surface and keeps sensitive services private.

Public access:

User Browser -> Frontend Public IP -> Nginx

Private access:

Frontend VM -> Backend VM -> Database VM

6. SSH Access Flow

A key pair was generated inside the frontend VM to access private backend and database VMs. The public key was added to the backend and database VMs using Azure Cloud Shell.

```
ssh-keygen -t ed25519 -f ~/.ssh/nexora_jump_key -N ""
```

```
az vm user update --resource-group rg-nexora-p26 --name vm-nexora-backend --username azureuser
--ssh-key-value "FRONTEND_PUBLIC_KEY"
```

```
az vm user update --resource-group rg-nexora-p26 --name vm-nexora-db --username azureuser --ssh-
key-value "FRONTEND_PUBLIC_KEY"
```

Laptop / Cloud Shell

|

v

Frontend VM

|

v

Backend VM and Database VM

7. Database Tier Deployment

7.1 Install PostgreSQL

```
sudo apt update
sudo apt install postgresql postgresql-contrib -y
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager
```

7.2 Create Database, User, and Product Table

```
sudo -u postgres psql -d nexora_db <<'EOF'
DROP TABLE IF EXISTS products;

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    category VARCHAR(50),
    image_url TEXT
);

INSERT INTO products (name, price, category, image_url) VALUES
('Wireless Headphones', 2999.00, 'Electronics', 'headphones'),
('Smart Watch', 4999.00, 'Electronics', 'watch'),
('Running Shoes', 2499.00, 'Fashion', 'shoes'),
('Backpack', 1499.00, 'Accessories', 'backpack'),
('Desk Lamp', 999.00, 'Home', 'lamp'),
('Bluetooth Speaker', 1999.00, 'Electronics', 'speaker');

SELECT * FROM products;
EOF
```

7.3 Allow Backend Access to PostgreSQL

```
sudo sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/"
/etc/postgresql/*/main/postgresql.conf

echo "host    nexora_db    nexora_user    10.26.2.0/24    md5" | sudo tee -a
/etc/postgresql/*/main/pg_hba.conf

sudo systemctl restart postgresql
sudo ss -tulnp | grep 5432
```

PostgreSQL was verified as listening on 0.0.0.0:5432, which allows private database access from the backend subnet.

7.4 Grant Application User Permissions

```
sudo -u postgres psql -d nexora_db <<'EOF'
GRANT CONNECT ON DATABASE nexora_db TO nexora_user;
GRANT USAGE ON SCHEMA public TO nexora_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO nexora_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO nexora_user;
EOF
```

8. Backend Tier Deployment

8.1 Install Runtime

```
sudo apt update
sudo apt install nodejs npm -y
node -v
npm -v
```

Verified runtime versions: Node.js v18.19.1 and npm 9.2.0.

8.2 Create Backend Application

```
mkdir -p ~/nexora-backend
cd ~/nexora-backend
npm init -y
npm install express pg cors dotenv
```

8.3 Backend Environment File

```
cat > .env <<'EOF'
DB_HOST=10.26.3.4
DB_USER=nexora_user
DB_PASSWORD=<DB_PASSWORD>
DB_NAME=nexora_db
PORT=5000
EOF
```

8.4 Backend API Code Summary

The backend application exposes three API endpoints:

Endpoint	Purpose
/api/health	Checks whether backend service is running
/api/db-test	Verifies backend-to-database connectivity
/api/products	Returns product records from PostgreSQL

8.5 Run Backend with PM2

```
sudo npm install -g pm2
cd ~/nexora-backend
pm2 start server.js --name nexora-backend
pm2 save
pm2 status
```

PM2 status verified the backend service as online.

9. Frontend Tier Deployment

9.1 Install Nginx

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
```

The Nexora frontend page was deployed to `/var/www/html/index.html`. It includes the homepage, product listing, login section, registration section, and API connection status message.

9.2 Nginx Reverse Proxy Configuration

```
server {
    listen 80;
    server_name _;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://10.26.2.5:5000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

sudo ln -sf /etc/nginx/sites-available/nexora /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
```

10. Final URLs and Verification

Item	URL / Result
Frontend Website	<code>http://<PUBLIC_IP></code>
Backend Health API	<code>http://<PUBLIC_IP>/api/health</code>
Database Test API	<code>http://<PUBLIC_IP>/api/db-test</code>
Product API	<code>http://<PUBLIC_IP>/api/products</code>

```
curl http://localhost/api/health
curl http://localhost/api/db-test
curl http://localhost/api/products
```

The final verification confirmed that the frontend is served by Nginx, the backend API is reachable through the Nginx reverse proxy, and the backend is able to fetch product data from PostgreSQL.

11. Screenshots Required for Submission

No.	Screenshot	Purpose
01	Resource group overview	Shows project resource group
02	VM list	Shows frontend, backend, and database VMs
03	Frontend VM overview	Shows public IP and private IP
04	Backend VM overview	Shows private IP
05	Database VM overview	Shows private IP
06	Nexora homepage	Shows public website
07	Product listing	Shows products loaded from backend API

08	Login and register section	Shows user interface requirement
09	/api/health	Shows backend health response
10	/api/db-test	Shows database connectivity response
11	/api/products	Shows database product data response

12. Troubleshooting Notes

Issue	Cause	Fix Applied
Frontend VM had no public IP	Public IP was not attached during VM creation	Created and associated a Standard static public IP
Backend package installation failed	Private VM had no outbound package access	Temporary public IP was used for installation
Product API permission denied	Database table permissions were missing	Granted table and sequence permissions to application user

13. Final Security Cleanup

After completing installation and screenshots, temporary public IPs must be removed from backend and database VMs. The final architecture should keep only the frontend VM public.

Final expected public exposure:
Frontend VM -> Public IP enabled
Backend VM -> No public IP
Database VM -> No public IP

14. Cost Control

After project review, VM compute billing can be stopped by deallocating the VMs.

```
az vm deallocate --resource-group rg-nexora-p26 --name vm-nexora-frontend
az vm deallocate --resource-group rg-nexora-p26 --name vm-nexora-backend
az vm deallocate --resource-group rg-nexora-p26 --name vm-nexora-db
```

To remove the complete project environment after final review:

```
az group delete --name rg-nexora-p26 --yes --no-wait
```

15. Project Completion Status

Component	Status
Frontend VM	Completed
Nginx web server	Running
Backend VM	Completed
Node.js API	Running with PM2
PostgreSQL database	Running
Backend-to-database connection	Working
Product API	Working
Public website	Working
Login/Register UI	Completed
Final private architecture cleanup	Pending verification